Technical Report

for

AflAlert

Prepared by

Group Name: GROUP 4

25/U/0350	NABACHWA PRISCILLA LUTAYA
25/U/03511/PS	NALUYIMA LAUREEN
25/U/03348/EVE	ATWINE SHEILA BAGUMIRE
25/U/03546/PSA	NSUBUGA BRIAN
25/U/26620	KIZITO JUMA

Mentor: MR. ASIIMWE PADDY

Course: CSC 1304 Practical Skills Development

Date: 1 August 2026

Contents

Abstract

AflAlert is a mobile-first, AI-assisted risk-detection application built to help maize farmers and grain handlers identify aflatoxin contamination risk without needing laboratory access. It combines two complementary detection tiers: an on-device image-classification model that flags visible mould/spoilage risk from a photo of maize kernels, and a chemical lateral-flow test-strip reader that estimates a contamination level in parts per billion (ppb) from a photographed, reacted strip.

Every scan is paired with guided next-step recommendations sourced from UNBS and MAAIF standards, alongside proactive rain/weather alerts that help farmers protect drying grain before it is exposed to moisture — a primary driver of aflatoxin growth.

AflAlert also provides scan history, exportable PDF reports, a hands-free voice assistant, and multilingual support (English and Luganda), giving smallholder farmers and grain handlers a practical, accessible tool for reducing contamination risk and protecting both health and income.

1 Introduction

This document presents the technical report for AflAlert, an aflatoxin risk-detection application developed as part of [Course Code and Title]. It provides an overview of the project's objectives, system design, functionality, implementation outcomes, limitations, and planned future enhancements.

The report serves as a formal record of the project's development process and achievements, detailing how AflAlert addresses the lack of accessible aflatoxin screening for farmers through on-device kernel scanning, chemical test-strip reading, guided recommendations, and proactive weather alerts.

1.1 User Challenge

Aflatoxin contamination in maize is a major food-safety and economic risk in many farming communities, but affected farmers and grain handlers rarely have access to laboratory testing. Visual inspection alone is unreliable — early or low-level contamination isn't visible to the eye — while commercial lab testing is costly, slow, and often geographically inaccessible for smallholder and rural sellers.

Meanwhile, unpredictable rain during the drying period is one of the most common causes of moisture re-uptake that drives mould and toxin growth; without timely local weather warnings, farmers often cannot cover or move drying grain in time. Grain handlers and farmers therefore need a fast, low-cost, easy first-line way to assess risk directly at the point of harvest or sale, along with practical guidance on what to do about a given result, and warning of conditions — like incoming rain — that raise their risk.

1.2 Project Goals

AflAlert's goal is to put a first-line aflatoxin risk check directly into the hands of farmers and grain handlers, using only a smartphone camera — no lab, no reagents beyond a commercially available test strip, no specialist training required.

It combines two complementary scanning tiers (kernel image analysis and strip-based chemical reading) so a user can get an assessment even without a strip on hand, and a more chemically-grounded reading when one is used. Every result is paired with actionable guidance rather than a bare number, and the app proactively protects against a leading cause of contamination — rain exposure during drying — through location-based weather alerts. Voice assistance, scan history, and shareable PDF reports round out the experience so results can be acted on, tracked over time, and shared with buyers, extension officers, or family members who help with drying and storage decisions.

1.3 Functional Requirements

1.3.1. Kernel Image Scanning

a) Capture or select a photo: Allow the user to photograph a sample of maize kernels via the in-app camera or pick an existing photo from the gallery.

b) On-device classification: Run the captured image through a bundled TensorFlow Lite model to classify the sample (e.g. healthy vs. mouldy/spoiled) and flag likely non-maize photos.

c) Confidence and rejection handling: Reject low-confidence or clearly non-maize images (colour-profile mismatch, model rejection, low confidence) rather than returning a misleading result.

1.3.2. Test Strip Scanning

a) Guided capture: Present a capture frame that guides the user to align a reacted lateral-flow test strip (Control line near the top, Test line near the bottom) and checks light/focus quality before use.

b) Strip plausibility check: Reject photos that don't plausibly resemble a strip cassette (e.g. not sufficiently pale/low-saturation) before attempting line detection.

c) Line detection: Locate the Control (C) and Test (T) line bands from a row-by-row luminance profile of the image, using a local-background baseline to stay robust to uneven lighting.

d) Optical density and ppb calculation: Compute the optical density of each line relative to its local background, derive a T/C ratio, and translate that ratio into an estimated contamination reading in ppb.

e) Validity (QC) checks: Flag a scan as invalid if no strip/lines are detected, or if the Control line isn't dark enough to indicate a properly developed run, rather than presenting a misleading low-ppb result.

1.3.3. Guided Recommendations

a) Result-linked guidance: Attach next-step guidance to every scan result (kernel or strip), sourced from UNBS and MAAIF standards, based on whether the result is within or above the safe threshold.

1.3.4. Weather and Rain Alerts

a) Daily morning summary: Provide a once-daily weather summary based on the user's location.

b) Urgent rain alerts: Independently monitor short-range forecasts and push an urgent alert when rain is imminent, so drying grain can be covered or moved in time, including while the app is closed (Android background scheduling).

1.3.5. Voice Assistant

a) Hands-free interaction: Allow the user to trigger scans and ask questions about results or recommendations using speech input and spoken responses.

b) Available across the app: Surface the assistant from the home, history, notifications, and results screens (kernel and strip), not just a single entry point, so it's reachable wherever a user might want it.

c) Spoken result summaries: When a scan is triggered from the assistant, speak a summary of the result (safe/unsafe classification) aloud via text-to-speech as soon as it's ready, using the same label logic as the on-screen result so the two never disagree.

1.3.6. History, Reports and Accounts

a) Scan history: Store and display a history of past kernel and strip scans per authenticated user.

b) PDF report export: Generate and share/download a PDF report for a given scan result.

c) Authentication: Support email/password and Google sign-in, OTP verification, password reset, and biometric app unlock.

d) Localization: Support English and Luganda throughout the app.

2 Project Results

2.1 Product Design

AflAlert is a cross-platform mobile application built with Flutter, using Firebase for authentication (email/password, Google Sign-In, OTP), real-time data storage (Cloud Firestore), file storage (Firebase Storage), server-side logic (Cloud Functions), and app integrity verification (Firebase App Check).

On-device machine learning is handled via tflite_flutter, running a bundled classification model for kernel scanning, while the test-strip reader uses deterministic pixel/image processing (the image package) rather than a trained model, since the T/C line-reading problem is well suited to direct optical-density calculation.

Background scheduling for rain/weather alerts uses WorkManager (Android) so alerts can fire even when the app isn't open. The codebase separates concerns cleanly into screens (UI), services (business logic — classification, strip analysis, Firebase access, alerts, PDF generation, voice), and localization resources, making each detection tier and supporting feature independently maintainable and testable.

2.1.1 Kernel Classification Model Training

The kernel classification model was trained on a maize image dataset curated and labelled on Roboflow (project “corn-aflatoxin-classification”, workspace priscilla-nabachwa), comprising 4,892 labelled images across three classes: Healthy (799 images), Moldy (1,348 images), and Non Maize (2,969 images) — the last of which teaches the model to reject photos that are not maize kernels at all, supporting the app's non-maize rejection check (see 1.3.1c).

Training was carried out in Google Colab using the Ultralytics YOLOv8 classification architecture (YOLOv8n-cls), starting from its pretrained weights and fine-tuned on the Roboflow dataset export — split by Roboflow into 9,960 training, 1,417 validation, and 717 test images across the three classes — for 25 epochs at a 224×224 input resolution. The resulting model was exported directly to the LiteRT (formerly TensorFlow Lite) format as a .tflite file, which is bundled into the app and run fully on-device via the tflite_flutter plugin (see 2.1) — keeping inference offline and avoiding any need to upload user photos to a server.

Training ran for 25 epochs (0.712 hours total on a Tesla T4 GPU) and converged to a final training loss of 0.0238. On the held-out validation split, the fused model (1,438,723 parameters, 3.3 GFLOPs) reached a top-1 accuracy of 98.8% and a top-5 accuracy of 100%, with an inference speed of roughly 0.5 ms per image on GPU.

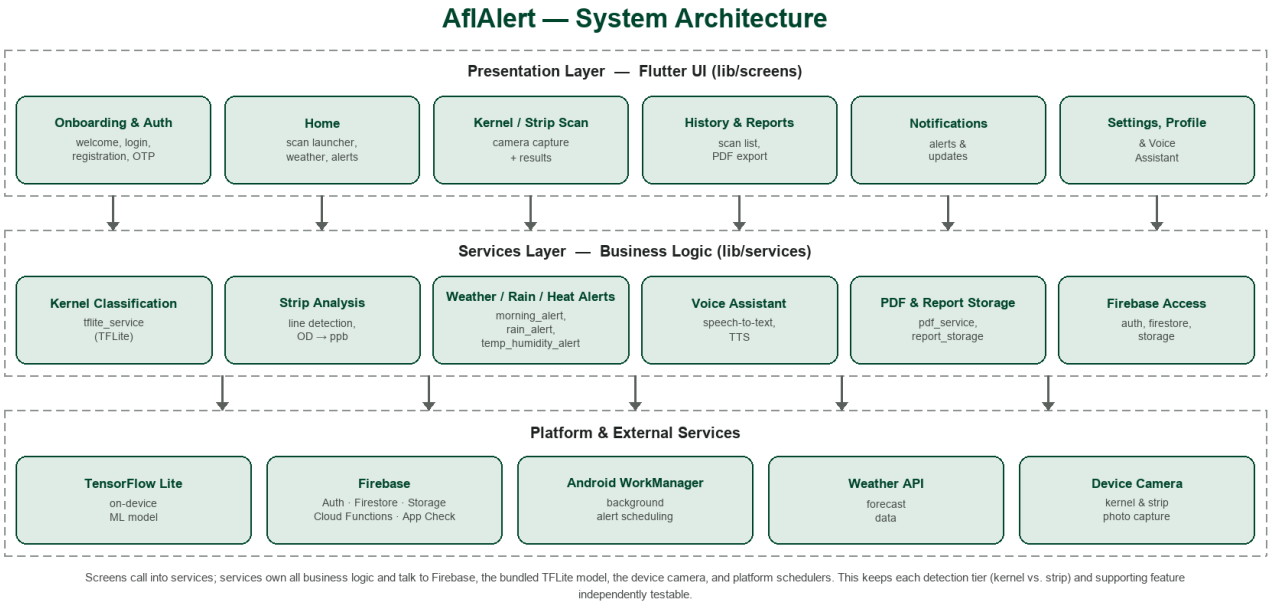
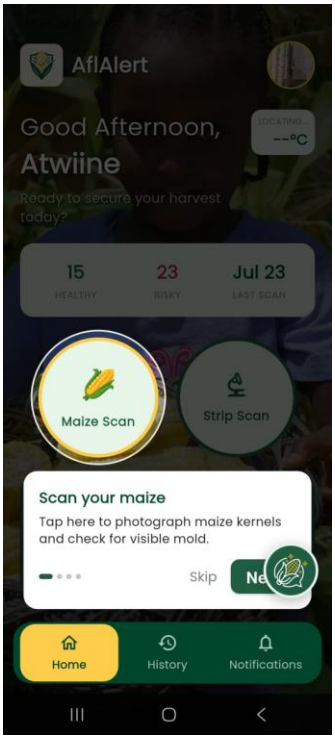


Figure 1: AflAlert system architecture — Flutter UI screens call into a services layer (classification, strip analysis, alerts, voice, PDF/report storage, Firebase access), which in turn talks to the platform and external services below it.

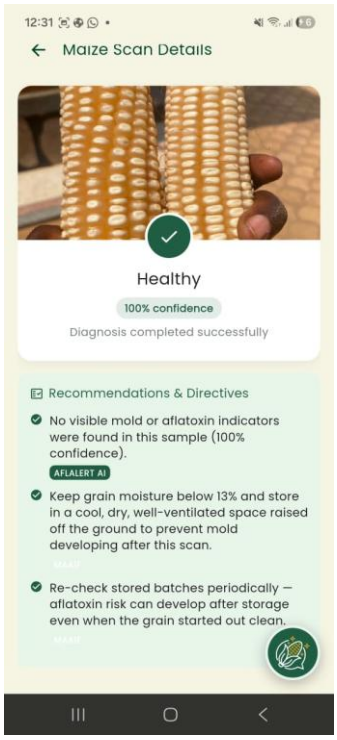
2.2 Product Functionality and Screenshots

a) Kernel Scan Functionality

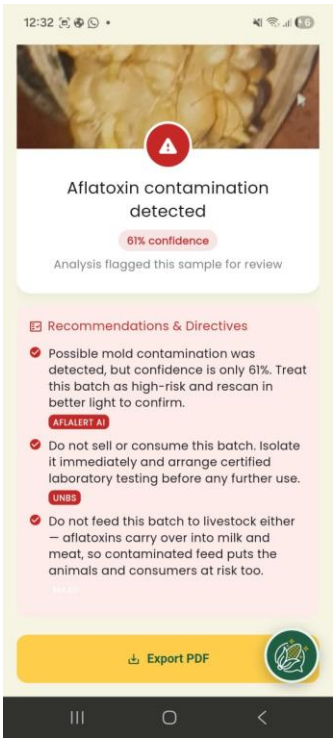
Users capture or select a photo of maize kernels; the app classifies the sample on-device and returns a risk result with guided recommendations.



Home screen — start a kernel scan



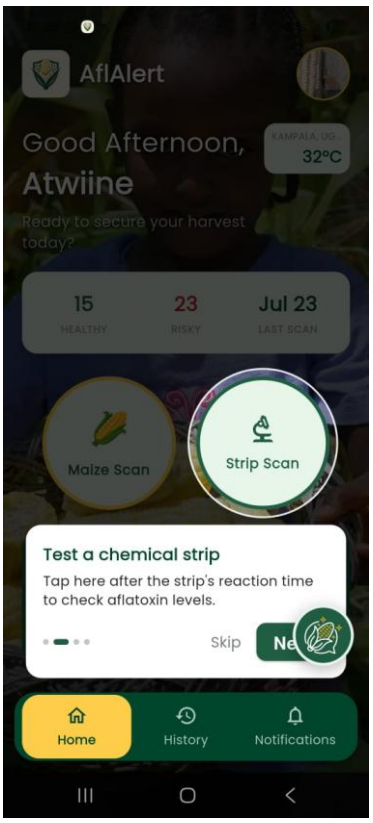
Result — Healthy (100% confidence)



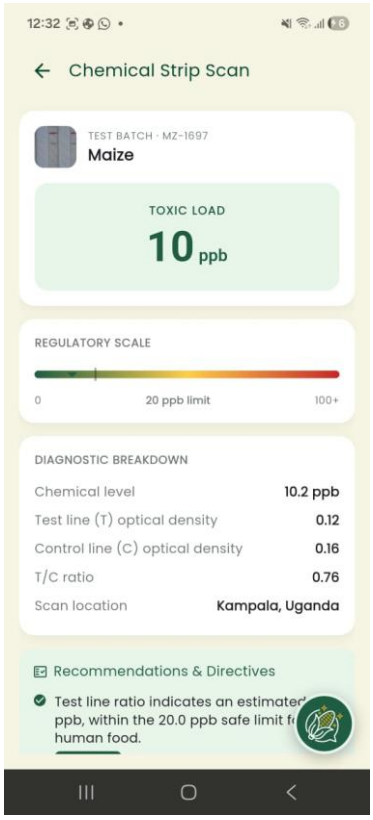
Result — contamination detected (61% confidence)

b) Test Strip Scan Functionality

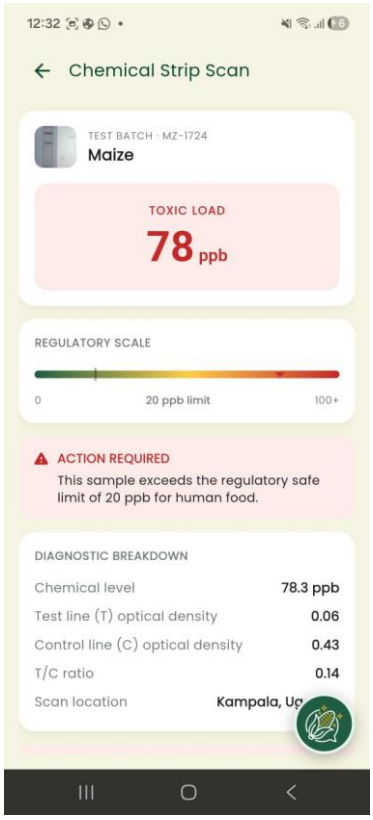
Users perform the physical extraction/strip test, then photograph the reacted strip through a guided capture flow; the app reads the Control and Test lines, computes an optical-density ratio, and returns an estimated ppb reading with a safe/unsafe classification and guidance.



Home screen — start a strip scan



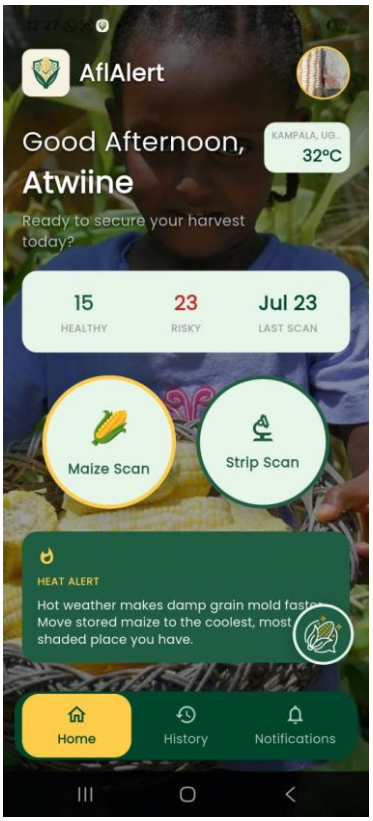
Result — 10 ppb, within the 20 ppb limit



Result — 78 ppb, action required

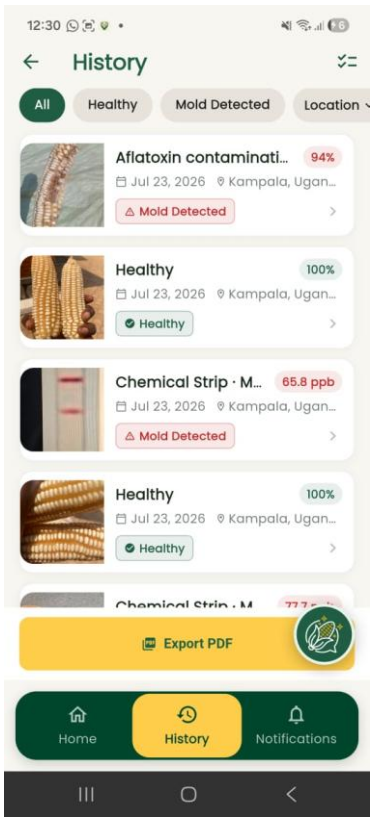
c) Alerts, History and Assistant Functionality

Users receive daily weather summaries and urgent rain alerts, review past scans and download PDF reports, and can interact with a voice assistant for hands-free scanning and Q&A. The assistant is reachable from the home, history, notifications, and results screens, and speaks a result summary aloud when a scan it triggered finishes.

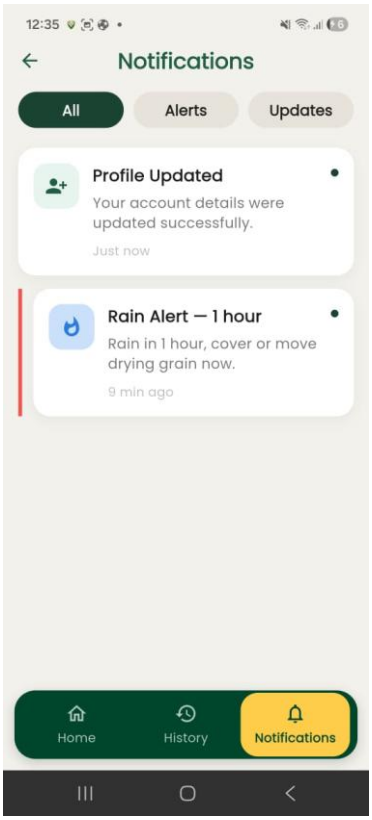


Heat alert on the home screen

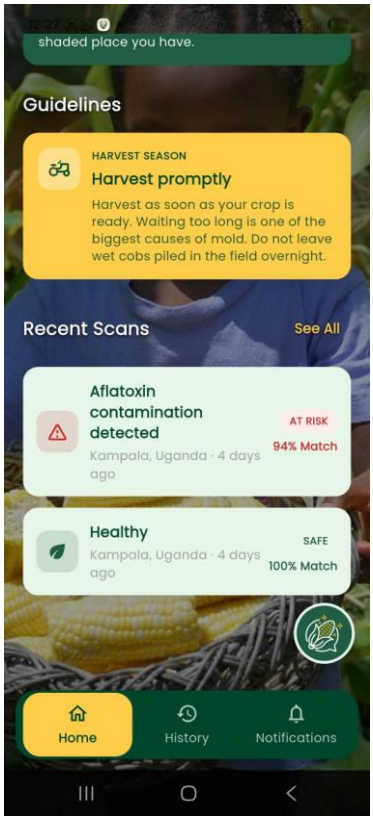
Voice assistant



Scan history



Notifications — alerts and updates



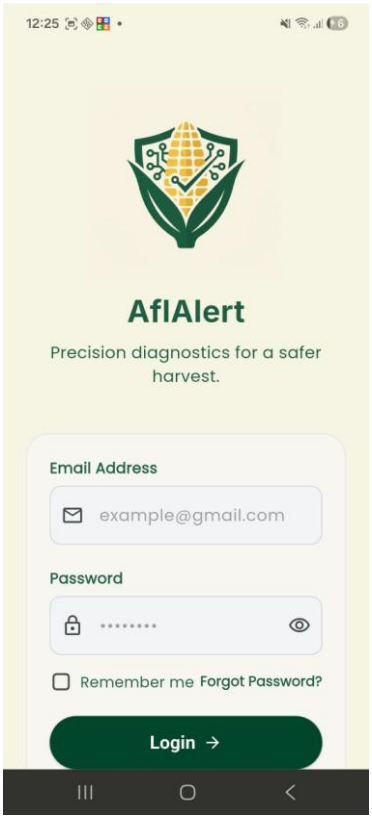
Home — guidelines and recent scans

d) Onboarding, Authentication and Account Functionality

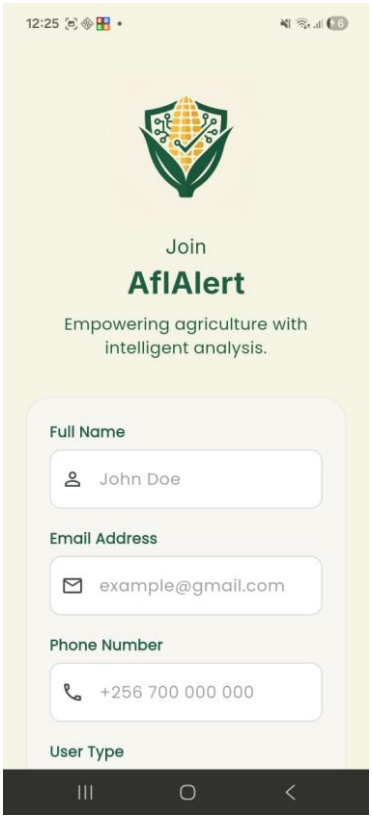
New users are guided through an onboarding carousel introducing the app's purpose, then register or log in (email/password or Google) with role and district captured at sign-up; the settings and profile screens expose language preference, notification toggles, and account management.

Onboarding carousel

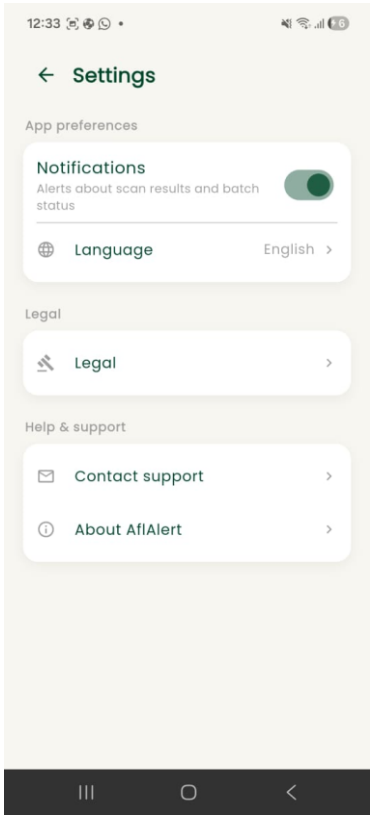
Welcome — sign up or log in



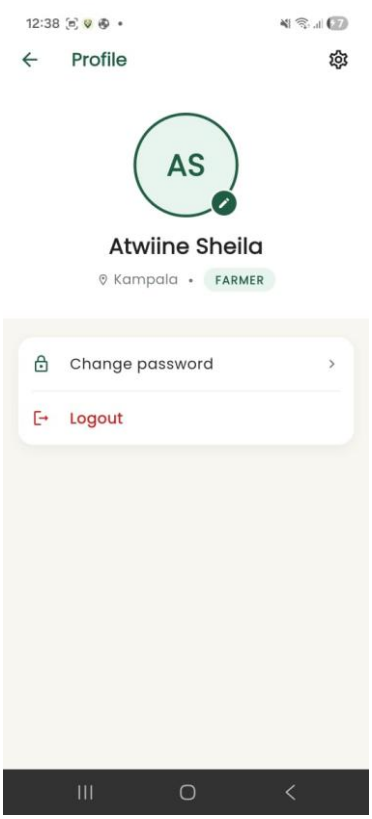
Login



Registration



Settings



Profile

2.3 Project Website and Repository

GitHub Repository Link: https://github.com/priscillanabachwa/AFLALERT

Project Website Link: https://naluyimalaureen-source.github.io/AFLALERT-website/

Running System Link: https://aflalert.web.app

Caution: the hosted web build above is provided to demonstrate onboarding, authentication, history, alerts, and the general UI live in a browser. Kernel and test-strip AI scanning depend on a bundled TensorFlow Lite model (tflite_flutter), which relies on dart:ffi and has no browser-compatible implementation, so camera-based scanning and classification are not available on the web build. These features are fully functional on the native Android and iOS builds.

3 Limitations and Next Steps

3.1 Limitations

a) Placeholder ppb calibration: The strip reader's ppb figure is currently derived from a monotonic placeholder curve rather than a lab-fitted calibration, so it should be treated as a relative/qualitative signal rather than a certified quantitative reading.

b) Gallery-picked strip photos are less reliable: Photos selected from the gallery (rather than the guided in-app camera capture) aren't cropped/oriented to the expected frame, making line detection less reliable on those images.

c) Background alerts are Android-only: iOS has no platform guarantee for background task timing, so the proactive rain/weather alert currently only runs reliably on Android.

d) No confirmatory lab pathway: The app offers a first-line risk read, not a certified lab-equivalent result, and does not yet route high-risk results to confirmatory testing.

3.2 Next Steps

a) Calibrate the strip reader: Run a dilution series of certified-reference aflatoxin standards through the strip and app pipeline, and fit a proper dose-response curve (e.g. 4-parameter logistic) to replace the placeholder ppb formula.

b) Improve gallery-photo robustness: Add stricter structural validation for non-guided (gallery) strip photos, or restrict strip scanning to in-app camera capture only.

c) iOS background alerts: Investigate iOS-appropriate scheduling (e.g. BGTaskScheduler) to bring rain alerts to iOS.

d) Confirmatory testing pathway: Add a recommended next step to route high-risk results toward certified lab confirmation.

References

[1] Google, "Flutter documentation," Flutter.dev. [Online]. Available: https://flutter.dev/docs.

[2] Google, "Firebase documentation," Firebase.google.com. [Online]. Available: https://firebase.google.com/docs.

[3] Dart team, "Dart language guides," Dart.dev. [Online]. Available: https://dart.dev/guides.

[4] Google, "TensorFlow Lite," TensorFlow.org. [Online]. Available: https://www.tensorflow.org/lite.

[5] "image," Dart package, pub.dev. [Online]. Available: https://pub.dev/packages/image.

[6] Codex Alimentarius Commission, "General standard for contaminants and toxins in food and feed," CODEX STAN 193-1995, FAO/WHO, Rome, Italy, 1995 (rev. 2019) — sets the international reference maximum levels for aflatoxins in food that UNBS/MAAIF grain guidance is aligned with.

[7] Uganda National Bureau of Standards (UNBS), "Maize grains — Specification," Kampala, Uganda. [Online]. Available: https://www.unbs.go.ug.

[8] Ministry of Agriculture, Animal Industry and Fisheries (MAAIF), "Post-harvest handling and grain storage guidelines," Kampala, Uganda. [Online]. Available: https://www.agriculture.go.ug.

[9] "flutter_tts," Dart package, pub.dev. [Online]. Available: https://pub.dev/packages/flutter_tts.

[10] "workmanager," Dart package, pub.dev. [Online]. Available: https://pub.dev/packages/workmanager.

Appendix A – Project Work Plan

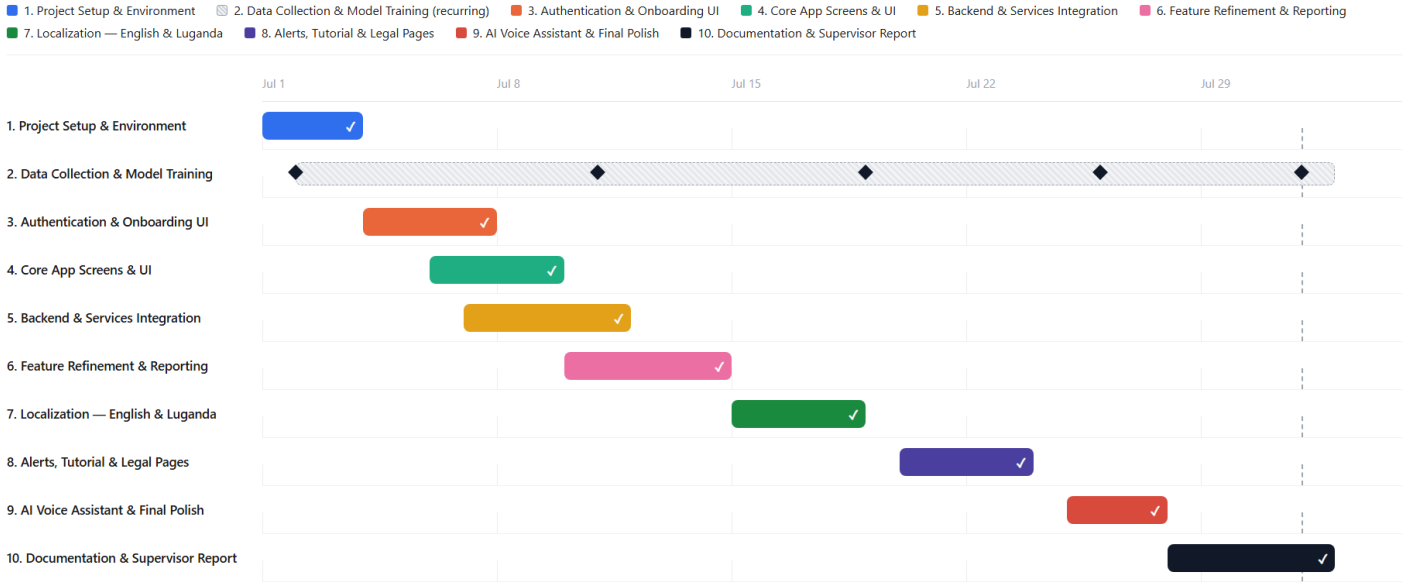
AFLALERT — Development Timeline

Mobile app for on-device maize aflatoxin risk detection · delivery schedule, Jul 1 – Aug 1, 2026 · prepared for supervisor review

Development window	Delivery phases	Contributors
32 days	**10**	**5**
Jul 1 – Aug 1, 2026	setup through final polish	parallel feature branches

Model iterations	Status
5	**Complete**
data collected & retrained each time	all phases merged to main

Project schedule



Phase details

Phase	Start	End	Duration	Key deliverables	Status
1. Project Setup & Environment	Jul 1	Jul 3	3 days	Initialized the Flutter project, configured Firebase (Core, Auth, Storage), integrated the first version of the best.tflite maize-disease detection model, and installed core dependencies (image_picker, tflite_flutter).	✓ Complete
2. Data Collection & Model Training	Jul 2	Aug 1	31 days (recurring)	Ongoing, parallel workstream: collected fresh field images of maize (healthy vs. aflatoxin-affected) and retrained the detection model each time best.tflite was updated — 5 retrain cycles shipped across the project, including an evening retrain run on Aug 1.	✓ 5 iterations shipped
3. Authentication & Onboarding UI	Jul 4	Jul 7	4 days	Built splash, welcome, login, registration and forgot-password screens; established shared widgets (app bar, bottom nav) and the app color theme.	✓ Complete
4. Core App Screens & UI	Jul 6	Jul 9	4 days	Built home, camera, analysis, results, history, notifications, profile and settings screens; unified navigation and visual style across the app.	✓ Complete
5. Backend & Services Integration	Jul 7	Jul 11	5 days	Wired Firebase Auth/Firestore/Storage, built camera, location and TFLite detection services; connected screens to live data and removed mock data.	✓ Complete
6. Feature Refinement & Reporting	Jul 10	Jul 14	5 days	Added PDF export, legal/guideline content, a dark-mode trial, and consistent color theming across screens.	✓ Complete
7. Localization — English & Luganda	Jul 15	Jul 18	4 days	Added flutter_localizations/intl, generated ARB translation files, and localized every screen (auth, home, camera, results, history, notifications, profile, settings).	✓ Complete
8. Alerts, Tutorial & Legal Pages	Jul 20	Jul 23	4 days	Built the weather-based alert system, an onboarding coach-mark tutorial, and Terms of Service / Privacy Policy pages; fixed navigation and rendering bugs.	✓ Complete
9. AI Voice Assistant & Final Polish	Jul 25	Jul 27	3 days	Integrated the AI voice assistant across the analysis, results and other screens; closed out final bugs on the history and forgot-password flows.	✓ Complete
10. Documentation & Supervisor Report	Jul 28	Aug 1	5 days	Compiled project documentation and the final development report for supervisor review.	✓ Complete

Appendix B – Contribution by Team Members

No.	Team Member	Contribution
1	NABACHWA PRISCILLA LUTAYA	✓ Configured Firebase and platform build setup (Android/iOS/Windows); resolved recurring build/run issues (Gradle, Kotlin) ✓ Built and iterated the home, history, and analysis screens ✓ Implemented image storage handling and maize image resizing for scans ✓ Worked on the forgot-password flow, settings screen, and legal section ✓ Contributed to localization of the login page and general UI
2	NALUYIMA LAUREEN	✓ Worked on iOS and Android platform configuration ✓ Built the login screen and created the forgot-password screen ✓ Added Google sign-in branding assets (logo sizing/placement) ✓ Contributed to the analysis and settings screens ✓ Added Luganda-language localization changes ✓ Resolved recurring merge conflicts during branch integration
3	ATWINE SHEILA BAGUMIRE	✓ Built out the home screen extensively, plus registration, login, and settings screens ✓ Implemented the history screen and alert notifications (weather-based alerts, daily tips) ✓ Added the voice assistant integration and its icon ✓ Wrote Terms of Service and legal/guideline content ✓ Localized major screens into English and Luganda (app_localizations) ✓ Led resolution of merge conflicts across the application branch integration

4	NSUBUGA BRIAN	✓ Set up localization support (flutter_localizations, intl dependencies) and initialized the Luganda ARB file ✓ Restructured the l10n folder and generated the English/Luganda localization delegate files ✓ Added the history screen ✓ Edited the profile screen and project dependencies (pubspec.yaml) ✓ Worked on platform-specific TFLite and Firebase Storage services ✓ Resolved merge conflicts by reconciling incoming changes
5	KIZITO JUMA	✓ Produced the Luganda translation file (app_lg.arb) and localization strings ✓ Built the notifications screen and result screen ✓ Worked on the welcome, login, and registration pages ✓ Sized and integrated the AflAlert logo across screens ✓ Integrated the TFLite/ML classification service and weather service ✓ Resolved merge conflicts on the home screen